PES UNIVERSITY

SOFTWARE ENGINEERING LABORATORY

LAB 7

FUZZING FOR INPUT VALIDATION BUGS

Student Name	Deeraj S
SRN	PES2UG25CS806
Course	Software Engineering
Date	11 November 2026

1. Objective

To apply fuzz testing to an Income Tax Calculator parser using the Hypothesis property-based testing library, identify input-validation bugs, improve the parser's handling of unexpected inputs, and verify that the corrected functions do not crash for generated test inputs.

2. Software and Tools Required

- Python 3.8 or later
- pytest
- Hypothesis
- Visual Studio Code or another Python editor
- Git and GitHub

3. Learning Outcomes

- Understand fuzz testing and property-based testing.
- Use Hypothesis strategies to generate a wide range of test inputs.
- Identify crashes caused by malformed, unexpected, or non-string input.
- Apply defensive input validation and safe error handling.
- Run the test suite and maintain the completed work in GitHub.

4. Introduction to Fuzz Testing

Fuzz testing is a dynamic software-testing technique in which a program is supplied with many generated or unexpected inputs to discover crashes, invalid assumptions, and incorrect behaviour. Instead of checking only a few manually selected examples, a fuzzing tool explores a much wider input space.

In this lab, Hypothesis generates text values and None values for the parser functions. The tests check that the functions handle these inputs without raising unhandled exceptions and return values of the expected type.

5. Application Under Test

The supplied application is an Income Tax Calculator parser containing three functions:

- clean_pan(pan): removes spaces from a PAN string and converts it to uppercase.
- parse_amount(text): converts a rupee amount such as 'Rs 12,50,000' into an integer.
- parse_tax_slab(line): parses a slab in LOW-HIGH:RATE format into a three-integer tuple.

6. Procedure Followed

1. Opened the LAB7 starter project and inspected income_tax_parser.py, test_income_tax_parser.py, and README.md.
2. Installed/verified pytest and Hypothesis in the Python environment.
3. Executed the application with representative PAN, amount, and tax-slab values.
4. Completed the Hypothesis tests for clean_pan, parse_amount, and parse_tax_slab.
5. Ran the tests against the initial parser and observed a failure in parse_amount for an unexpected generated input.
6. Updated the parser with type checks, input validation, and safe handling of invalid values.
7. Re-ran the Hypothesis tests and confirmed that all three tests passed.
8. Committed and pushed the final parser update to the combined GitHub repository.

7. Bug Identified and Fix Applied

The initial parse_amount implementation used str.isdigit() to validate the cleaned amount and then called int(). Hypothesis discovered an input containing a character that passed the digit check but could not be converted by int(), causing a ValueError.

The corrected implementation validates that every remaining character is an ASCII digit (0–9) before conversion and catches conversion errors. This prevents the discovered input from causing an unhandled crash.

Before	After
Used text.isdigit() followed by int(text), which could fail for some Unicode digit characters.	Checks each character against '0123456789' and catches ValueError/OverflowError.

8. Test Design Using Hypothesis

Each test uses the @given decorator and st.one_of(st.text(), st.none()) to generate string inputs and None. The property checked is that each parser function completes without an unhandled exception and returns the expected type.

```
from hypothesis import given, strategies as st
from income_tax_parser import clean_pan, parse_amount, parse_tax_slab


@given(st.one_of(st.text(), st.none()))
def test_clean_pan_no_crash(s):
    result = clean_pan(s)
    assert isinstance(result, str)


@given(st.one_of(st.text(), st.none()))
def test_parse_amount_no_crash(s):
    result = parse_amount(s)
    assert result is None or isinstance(result, int)


@given(st.one_of(st.text(), st.none()))
def test_parse_tax_slab_no_crash(s):
    result = parse_tax_slab(s)
    assert result is None or (
        isinstance(result, tuple)
        and len(result) == 3
        and all(isinstance(value, int) for value in result)
    )
```

9. Final Test Execution and Result

Command executed:

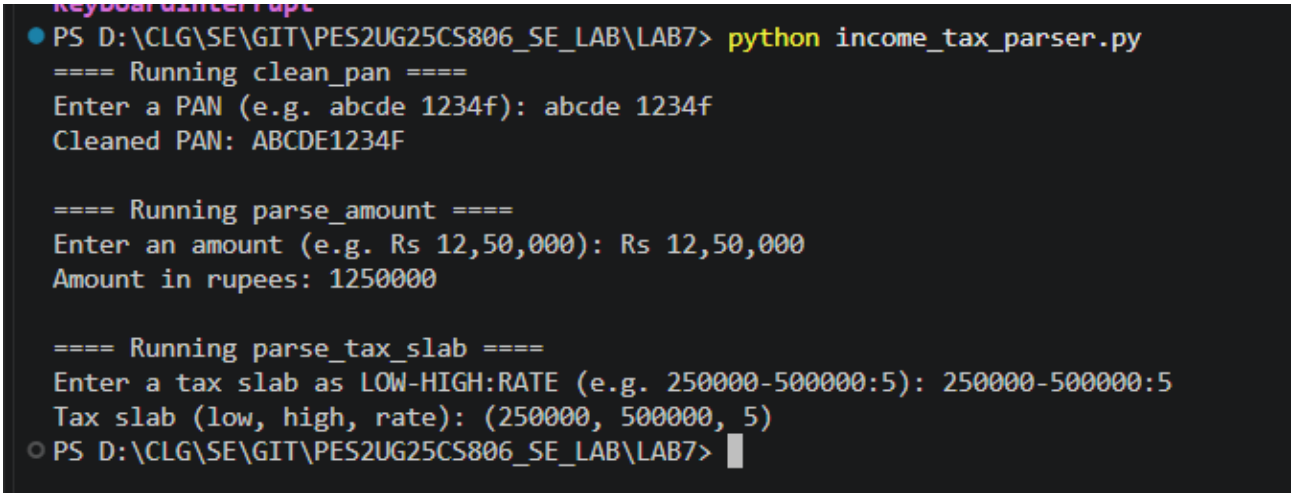
```
python -m pytest -v test_income_tax_parser.py
```

The final run collected three tests. All three tests passed, confirming that the corrected parser functions handled the generated test inputs without unhandled crashes.

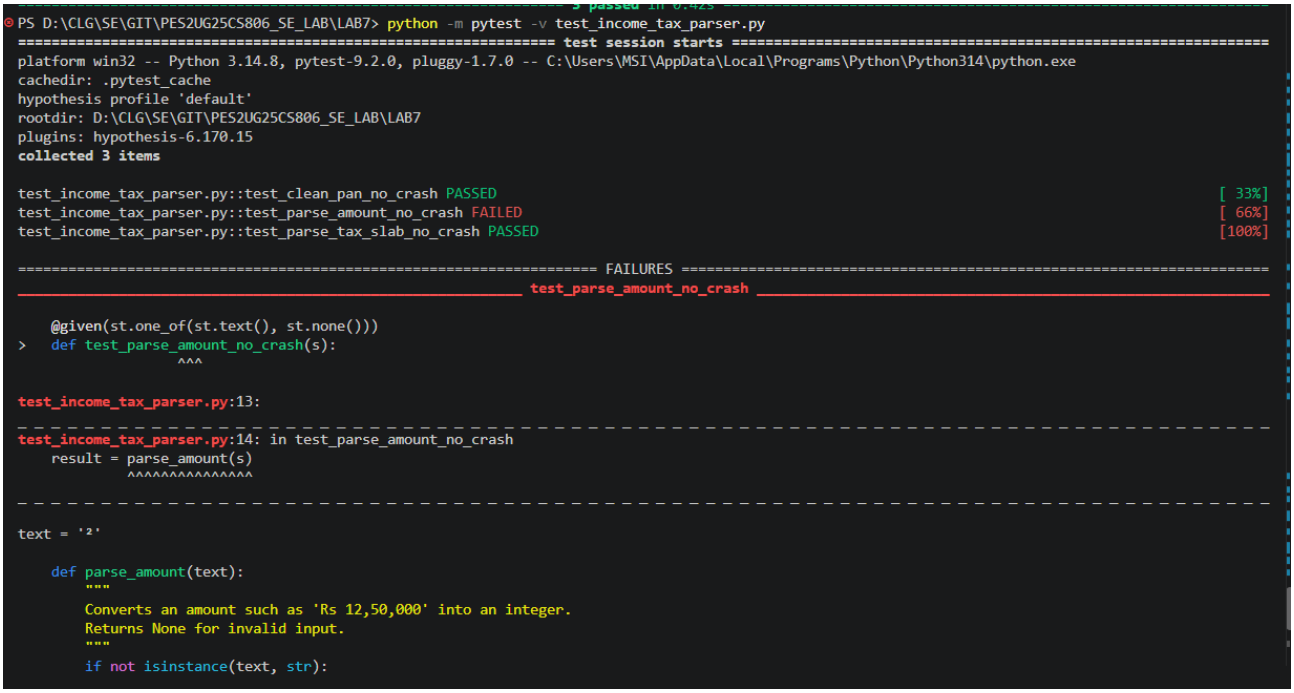
Test	Final status
test_clean_pan_no_crash	**PASSED**
test_parse_amount_no_crash	**PASSED**
test_parse_tax_slab_no_crash	**PASSED**
Overall result	**3 passed, 0 failed**

10. Screenshot Evidence

The following evidence images were captured during the lab work. SS1 shows the initial application run, SS2 shows the Hypothesis failure on the buggy parser, SS3 shows the corrected parse_amount implementation in the editor, and SS4 shows the final successful test run.



SS1 – Initial Income Tax Calculator parser execution with sample inputs.



SS2 – Hypothesis test failure on the initial parse_amount implementation.

```
    return pan.strip().replace(" ", "").upper()



def parse_amount(text):
    """
    Converts an amount such as 'Rs 12,50,000' into an integer.
    Returns None for invalid input.
    """
    if not isinstance(text, str):
        return None

    text = text.strip()
    text = text.replace("Rs", "").replace("rs", "")
    text = text.replace(",", "").replace(" ", "")

    if not text or not all(char in "0123456789" for char in text):
        return None

    try:
        return int(text)
    except (ValueError, OverflowError):
        return None



def parse_tax_slab(line):
    """
    Parses 'LOW-HIGH:RATE' into a (low, high, rate) tuple.
```

SS3 – Corrected parse_amount implementation in the source editor.

```
PS D:\CLG\SE\GIT\PES2UG25CS806_SE_LAB\LAB7> python -m pytest -v test_income_tax_parser.py
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: D:\CLG\SE\GIT\PES2UG25CS806_SE_LAB\LAB7
plugins: hypothesis-6.170.15
collected 3 items

test_income_tax_parser.py::test_clean_pan_no_crash PASSED                [ 33%]
test_income_tax_parser.py::test_parse_amount_no_crash PASSED             [ 66%]
test_income_tax_parser.py::test_parse_tax_slab_no_crash PASSED           [100%]

============================== 3 passed in 0.72s ==============================
PS D:\CLG\SE\GIT\PES2UG25CS806_SE_LAB\LAB7>
```

SS4 – Final pytest execution: all three tests passed.

Evidence note: The available SS3 image shows the corrected source code in the editor, not a post-fix terminal execution of python income_tax_parser.py. If the lab evaluator strictly requires SS3 to show the fixed module being run, capture that terminal run and replace this SS3 image before submission.

11. Complete Corrected Application Code

```
def clean_pan(pan):
    """
    Removes spaces from a PAN and converts it to uppercase.
    Returns an empty string for invalid input.
    """
    if not isinstance(pan, str):
        return ""

    return pan.strip().replace(" ", "").upper()


def parse_amount(text):
    """
```

```
    Converts an amount such as 'Rs 12,50,000' into an integer.
    Returns None for invalid input.
    """
    if not isinstance(text, str):
        return None

    text = text.strip()

    if not text:
        return None

    text = text.replace("Rs", "").replace("rs", "")
    text = text.replace(",", "").replace(" ", "").strip()

    if not text or not all(char in "0123456789" for char in text):
        return None

    try:
        return int(text)
    except (ValueError, OverflowError):
        return None


def parse_tax_slab(line):
    """
    Parses 'LOW-HIGH:RATE' into a (low, high, rate) tuple.
    Returns None for invalid input.
    """
    if not isinstance(line, str):
        return None

    try:
        limits, rate = line.split(":")
        low, high = limits.split("-")

        low = low.strip()
        high = high.strip()
        rate = rate.strip()

        if not all(value and all(char in "0123456789" for char in value)
                   for value in (low, high, rate)):
            return None

        low = int(low)
        high = int(high)
        rate = int(rate)

        if low > high:
            return None

        return low, high, rate

    except (ValueError, AttributeError, OverflowError):
        return None


def main():
    print("==== Running clean_pan ====")
    value = input("Enter a PAN (e.g. abcde 1234f): ")
    print("Cleaned PAN:", clean_pan(value))

    print("\n==== Running parse_amount ====")
    value = input("Enter an amount (e.g. Rs 12,50,000): ")
    result = parse_amount(value)
    print("Amount in rupees:", result)

    print("\n==== Running parse_tax_slab ====")
    value = input(
```

```
        "Enter a tax slab as LOW-HIGH:RATE "
        "(e.g. 250000-500000:5): "
    )
    result = parse_tax_slab(value)
    print("Tax slab (low, high, rate):", result)


if __name__ == "__main__":
    main()
```

12. GitHub Submission

Combined repository:

https://github.com/DEERAJSCoding/PES2UG25CS806_SE_LAB

Lab 7 folder:

https://github.com/DEERAJSCoding/PES2UG25CS806_SE_LAB/tree/main/LAB7

The final parser update was committed and pushed to the main branch. The local Git status confirmed that the branch was up to date with origin/main; an unused backup file remained untracked and was not included in the commit.

13. Conclusion

Fuzz testing was successfully applied to the Income Tax Calculator parser using Hypothesis. The generated tests exposed an input-validation weakness in parse_amount. The parser was strengthened with explicit input checks and safe conversion handling. The final test run reported 3 passed and 0 failed, demonstrating improved robustness for the tested input space.